

Actividad 4

Alumno

Alan Lam Páez

Matrícula

T03063809

Materia

Aplicaciones Interactivas

Maestro(a)

Cristopher Gerardo Gaytán Díaz

Correo

al03063809@tecmilenio.mx

Fecha

21/09/2026

Introducción a Unreal Engine

- **¿Qué es Unreal Engine?**

Unreal Engine (UE) es un motor de desarrollo de videojuegos 3D desarrollado por Epic Games desde 1995, cuyo creador y principal desarrollador inicial fue Tim Sweeney. UE proporciona herramientas para gráficos 3D, físicas, iluminación, sonido, animaciones entrada del jugador, inteligencia artificial y programación de gameplay.

Actualmente, un punto atractivo del motor es que permite programar juegos mediante *Blueprint Visual Scripting*, el cuál crea lógica mediante nodos visuales en lugar describir código manualmente en C++. Sin embargo, Epic Games anunció recientemente que existen planes de volver a los Blueprints obsoletos, optando mejor por un lenguaje de programación llamado *Verse* que principalmente se utiliza en Unreal Editor for Fortnite (UEFN).

- **¿Qué es un Actor?** Un actor funciona como un objeto que puede existir dentro de un nivel de UE. Un actor puede ser: un personaje, una puerta, cámara, luz, etc. Pueden ser creados, modificados y destruidos durante el juego. Se pueden crear como blueprints, y normalmente están formados por Components que proporcionan diferentes características.
- **¿Qué es un Blueprint?** Un Blueprint es un sistema de programación visual de Unreal Engine el cuál permite crear lógica y comportamiento de un objeto utilizando nodos conectados por cables. Un Blueprint Class (como un Actor Blueprint) puede utilizar variables y lógica, y del crearse varias instancias. Se asemeja a la programación orientada a objetos.
- **¿Qué es un Component?** Un component es una pieza de funcionalidad que se puede añadir a un Actor. Por ejemplo, un personaje controlado por el jugador puede tener un Static Mesh (el cuerpo), un Skeletal Mesh (el esqueleto para animar el cuerpo), Sphere Collision (colisión), Movement Component (gravedad).

- **¿Qué función tiene el Event Graph?** El Event Graph es el lugar dentro de un Blueprint donde normalmente se construye toda la lógica. Aquí se arrastran funciones, variables, custom events...en adelante. El tipo de Graph o espacio depende de lo que quieras hacer. Normalmente, las funciones, locomotion, etc., están separadas por varios espacios donde utilizas diferentes nodos. Es decir, un Animation Graph no es lo mismo que un Event Graph por su propósito.
- **¿Qué son los eventos?** Los events (eventos) son puntos de entrada que hacen que una determinada parte de la lógica de un Blueprint comience a ejecutarse cuando ocurre algo. Por ejemplo, existe el Begin Play cuando el juego comienza para ese actor, o un Tick que es código que se ejecuta cada cuadro por segundo (frame per second).
- **¿Qué son las funciones?** Las funciones son un bloque reutilizable de lógic. En lugar de escribir la misma secuencia de nodos varias veces, puedes colocarla dentro de una función y después llamar a esa función cuando la necesites.
- **¿Qué son las variables?** Una Variable es un espacio que almacena un valor (boolean, string, números) o una referencia que tu Blueprint puede utilizar (por ej., object reference).

Relación entre POO y Unreal Engine

Epic describe las clases de Blueprint como una forma de definir clases y objetos orientados a objetos.¹ Una forma de verlo es:

Concepto de POO	Definición	Aplicación en Unreal Engine	Ejemplo
Clase	Es una plantilla que define las propiedades y comportamientos que tendrán los objetos que se creen a partir de ella.	En Unreal, las clases definen tipos de Actors y Objects . Se pueden crear mediante C++ o Blueprints.	BP_Enemy puede definir cómo es y cómo se comporta un enemigo.

¹ <https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-terminology>

Objeto	Es una instancia concreta de una clase. Mientras la clase es la plantilla, el objeto es una instancia creada a partir de ella.	Los objetos de Unreal derivan de UObject. Los Actors son objetos que pueden existir en un nivel.	Si BP_Enemy es la clase, un enemigo específico colocado en el nivel es una instancia de esa clase.
Atributo	Es un dato que pertenece a un objeto y representa alguna de sus características o su estado.	En Unreal se representa principalmente mediante variables o propiedades de una clase.	Un enemigo puede tener Health = 100, Speed = 500 y IsAlert = true.
Método	Es una función asociada a una clase u objeto que define una acción o comportamiento que puede realizar.	Unreal utiliza funciones en C++ y Blueprints para implementar comportamientos.	Una función TakeDamage() puede reducir la vida de un enemigo.
Herencia	Permite crear una clase nueva a partir de otra, heredando sus propiedades y comportamientos y pudiendo agregar o modificar funcionalidades.	Las clases de Unreal tienen una jerarquía. Un Blueprint puede utilizar otra clase como Parent Class y heredar sus características.	BP_Dog puede heredar de BP_Animal, obteniendo sus variables y funciones y agregando comportamientos propios.

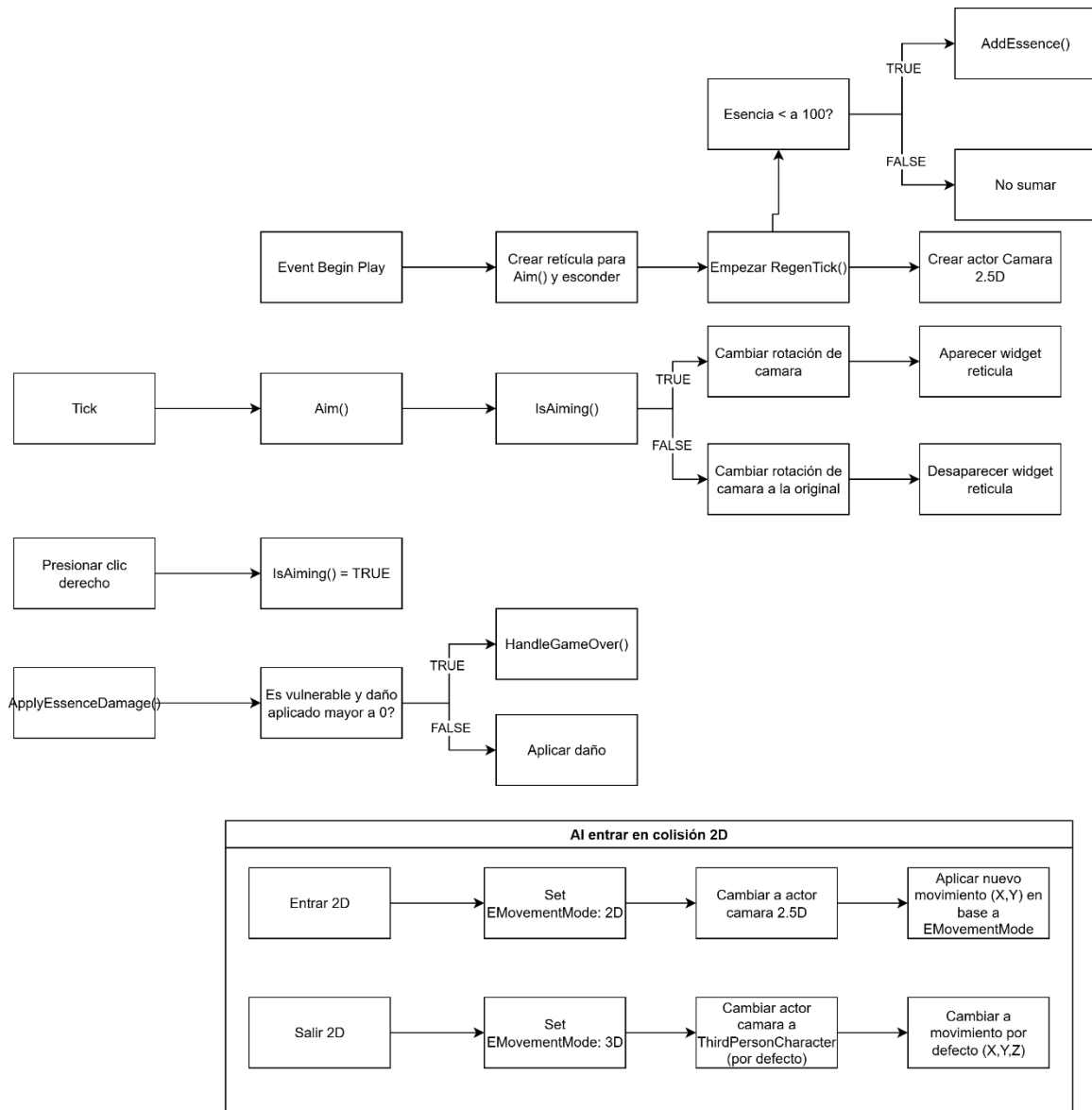
Análisis de un personaje como objeto

Personaje: Cosmo (proyecto personal)

Atributos	• Essence (float): la vida/magia del personaje. • EssenceMax (float): la máxima vida/magia que puede tener el personaje. • RegenRate (Int): el intervalo con el que el personaje recupera esencia. • IsVulnerable (boolean): si se encuentra vulnerable a un ataque que le llevara al fin de la partida. • EMovementMode (Enum): si se moverá en X,Y,Z (3D) o en X,Y (2D). • IsAiming (boolean): si esta enfocando con clic derecho.
------------------	---

	• IsGameOver (boolean): si el personaje perdió su esencia y es fin de la partida.
Métodos	• AddEssence(): Añadir esencia la variable Essence. • ApplyEssenceDamage(): Restar esencia a la variable Essence: • RegenTick(): Programar el intervalo para regenerar Esencia. • Aim(): Activar el enfoque y mover la cámara para dejar al actor a un lado. • HandleGameOver(): Arrancar pantalla de GameOver y no mover el personaje.

Diseño conceptual de un Blueprint

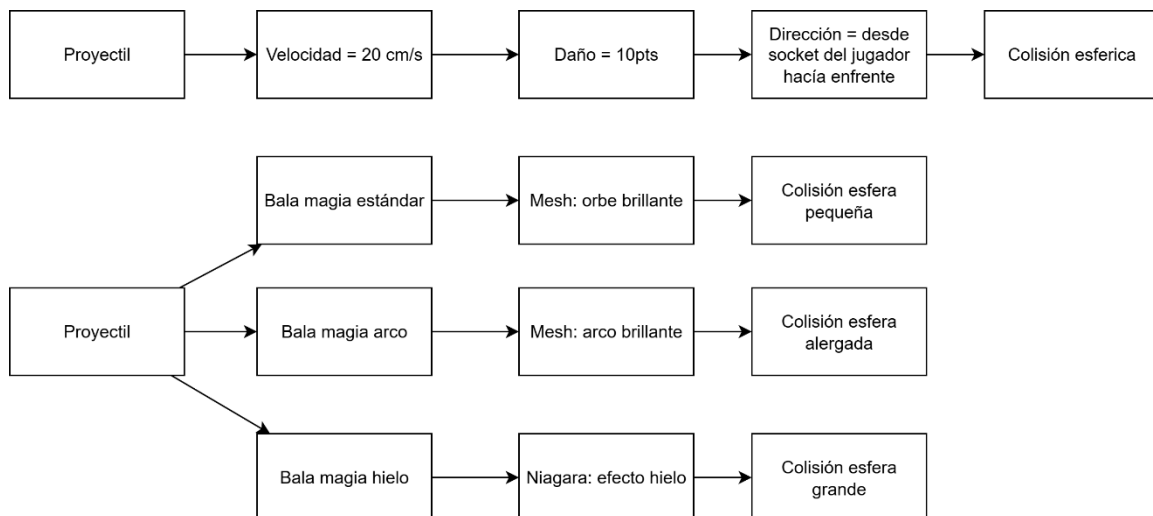


Lógica de eventos (ejemplos)

- Empezar RegenTick(): la esencia actual del actor es menos que 100?
 - TRUE = añadir esencia (AddEssence)
 - FALSE = no añadir esencia
- El jugador esta en una colisión 2D?

- TRUE = Entrar a 2D > cambiar MovementMode a 2D > Cambiar a actor camara 2.5D > Cambiar control de movimiento
- FALSE = Salir 2D > cambiar MovementMode a 3D > cambiar MovementMode a 3D > Cambiar actor a camara de ThirdPersonCharacter (actor actual) > Cambiar movimiento al original (X,Y,Z)
- ¿El jugador es vulnerable? > Desde ApplyEssenceDamage() > IsVulnerable == True & daño aplicado mayor a 0
 - TRUE = Game Over
 - False = Aplicar daño

Investigación sobre la herencia



¿Qué es la herencia en POO? La herencia es un mecanismo de la programación orientada a objetos que permite crear una clase nueva a partir de otra existente. La clase hija hereda las características y comportamientos de la clase padre y puede añadir características propias o modificar algunos comportamientos. En un videojuego, permite crear diferentes tipos de objetos que comparten una estructura y comportamiento básico, sin tener que programar desde cero las características que tienen en común.

¿Qué características comparten? Todos los proyectiles necesitan características y comportamientos básicos similares. Por ejemplo, todos tienen una posición, una dirección, una velocidad, un daño y deben poder

detectar una colisión. También necesitan algún comportamiento cuando impactan contra otro objeto.

¿Qué características son diferentes? Cada tipo de proyectil puede tener diferentes valores y comportamientos. Una bala puede ser rápida y causar poco daño, mientras que un cohete puede ser más lento, causar más daño y generar una explosión. La bola de hielo puede causar daño y, además, ralentizar al enemigo.

¿Qué elementos deberían pertenecer a la clase padre? En Proyectil deberían colocarse los elementos que sean comunes a todos los proyectiles, como:

- Velocidad.
- Daño.
- Dirección.
- Movimiento.
- Detección de colisiones.
- Comportamiento básico al impactar.

De esta manera, las clases hijas pueden reutilizar esa funcionalidad.

¿Qué elementos deberían pertenecer a las clases hijas? Las clases hijas deberían contener las características específicas que solamente correspondan a ese tipo de proyectil.

¿Qué ventaja proporciona la herencia? La principal ventaja es la reutilización de código. Las características comunes solamente necesitan implementarse una vez en la clase padre y pueden ser heredadas por todas las clases hijas. Esto reduce la duplicación de código y facilita modificar o ampliar el juego.

Por ejemplo, si posteriormente se cambia la forma en que todos los proyectiles detectan las colisiones, se puede modificar la implementación en Proyectil y las clases hijas pueden aprovechar ese cambio sin tener que implementar nuevamente la misma lógica.

Bibliografía

Academy of Interactive Arts & Sciences. (7 de diciembre de 2011). Academy of Interactive Arts & Sciences inducts Epic Games' Tim Sweeney to its Hall of Fame. Recuperado de

https://www.interactive.org/images/press_release/AIAS_Hall_of_Fame_Tim_Sweeney_FINAL.pdf

Beyond3D. (24 de febrero de 2004). Tim Sweeney of Epic Games. Recuperado de <https://www.beyond3d.com/content/interviews/18/1>

Edwards, B. (25 de mayo de 2009). From the past to the future: Tim Sweeney talks. Game Developer. Recuperado de

<https://www.gamedeveloper.com/design/from-the-past-to-the-future-tim-sweeney-talks>

Epic Games. (s. f.). About Epic Games. Recuperado de

<https://www.epicgames.com/site/about?lang=en-US>

Epic Games. (s. f.). Actors in Unreal Engine. Epic Developer Community.

Recuperado de <https://dev.epicgames.com/documentation/en-us/unreal-engine/actors-in-unreal-engine>

Epic Games. (s. f.). Anatomy of a Blueprint in Unreal Engine. Epic Developer Community. Recuperado de <https://dev.epicgames.com/documentation/en-us/unreal-engine/anatomy-of-a-blueprint-in-unreal-engine>

Epic Games. (s. f.). Basic components in Unreal Engine. Epic Developer

Community. Recuperado de <https://dev.epicgames.com/documentation/en-us/unreal-engine/basic-components-in-unreal-engine>

Epic Games. (s. f.). Basic scripting with Blueprints in Unreal Engine. Epic Developer Community. Recuperado de

<https://dev.epicgames.com/documentation/en-us/unreal-engine/basic-scripting-with-blueprints-in-unreal-engine>

Epic Games. (s. f.). Blueprint class assets in Unreal Engine. Epic Developer Community. Recuperado de <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-class-assets-in-unreal-engine>

Epic Games. (s. f.). Blueprint foundations. Epic Developer Community. Recuperado de <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-foundations>

Epic Games. (s. f.). Graphs in Unreal Engine. Epic Developer Community. Recuperado de <https://dev.epicgames.com/documentation/en-us/unreal-engine/graphs-in-unreal-engine>

Epic Games. (s. f.). Overview of Blueprints visual scripting in Unreal Engine. Epic Developer Community. Recuperado de <https://dev.epicgames.com/documentation/en-us/unreal-engine/overview-of-blueprints-visual-scripting-in-unreal-engine>

Epic Games. (s. f.). Types of Blueprints in Unreal Engine. Epic Developer Community. Recuperado de <https://dev.epicgames.com/documentation/en-us/unreal-engine/types-of-blueprints-in-unreal-engine>

Martínez, R. (29 de noviembre de 2021). De Unreal a Fortnite: la historia de Epic Games. MeriStation. Recuperado de https://as.com/meristation/2021/11/29/reportajes/1638173667_712771.html